PROJECT REPORT

OF

SMART CITY NAVIGATION SYSTEM

A Hybrid Web Application: Real-Map Routing + Graph Algorithm Lab

UNIVERSITY INSTITUTE OF ENGINEERING

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SUBMITTED BY:

Akshay Boora(25bcs11028)

Kartik(25bcs12387)

Kul Mahajan(25bcs12381)

Section/Group: 25BCS-802(A)

Semester: 3rd

Program: B.E CSE Core

SUBMITTED TO:

Mr. MD Tauseef

Subject: Advance Data Structure

Subject Code: 25CSH-202

CERTIFICATE

This is to certify that the project entitled "***SMART CITY NAVIGATION SYSTEM***" has been prepared and submitted by ***Akshay Boora(25bcs11028) , Kartik(25bcs12387) and Kul Mahajan(25bcs12381)*** as part of the academic requirements for the subject Advance Data Structure (Subject Code: 25CSH-202), Semester 3rd, Session July–December 2026.

The project demonstrates the use of graph, priority queue (min-heap), queue and stack data structures through a hybrid web application that combines real-map route planning with an Algorithm Lab implementing Dijkstra's algorithm, Breadth-First Search and DepthFirst Search, including road-closure re-routing. The work is presented for academic evaluation.

We wish the student success in future academic and professional endeavours.

Signature of Project Guide

_____

Mr. MD Tauseef

Technical Trainer

University Institute of Computer Science &

Engineering Chandigarh University

Date: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Mr. MD Tauseef sir(E19861), Technical Trainer, for his valuable guidance and support throughout the completion of the project, “SMART CITY NAVIGATION SYSTEM.”

I am also thankful to Chandigarh University for providing the opportunity and resources required to complete this project as part of the subject Advance Data Structure (Subject Code: 25CSH-202), Semester 3rd, Session July–December 2026.

Finally, I would like to acknowledge everyone whose support and encouragement helped me complete this project successfully.

ABSTRACT

This project presents the design and implementation of a Smart City Navigation System using Python (Flask), JavaScript and SQLite. It has two clearly separated parts. The ***Map Navigation*** module lets a user search real places, obtain real driving routes with alternatives, and compare distance and estimated travel time on an OpenStreetMap-based Leaflet map. The ***Algorithm Lab*** is an educational simulation in which our own implementations of Dijkstra's algorithm, BFS and DFS run on a 9-location, 14-road sample graph with step-by-step playback.

A city is modelled as a weighted, undirected graph stored as an adjacency list. Dijkstra's algorithm uses a binary min-heap, BFS uses a queue and DFS uses an explicit stack. Roads can be closed interactively; the algorithms are re-run on the changed graph, a before/after comparison is shown, and disconnected locations are detected with connected components. A rule-based assistant explains results in plain language, and SQLite stores history, favourites and analytics.

The project demonstrates the practical application of graph data structures and shortest-path algorithms in navigation, and is deployed online at https://smart-city-navigation-wwkj.onrender.com/?utm_source=chatgpt.com#map

INDEX

1. INTRODUCTION 6

2. PROBLEM STATEMENTS 6

3. OBJECTIVES 6

4. DATA STRUCTURE SELECTION AND JUSTIFICATION 8

5. ALGORITHM AND FLOWCHART 9

6. WORKING IMPLEMENTATION (PYTHON SOURCE CODE) 10

7. SAMPLE INPUT AND OUTPUT 13

8. TIME COMPLEXITY ANALYSIS 15

9. PROJECT REPORT AND DEMONSTRATION 15

10. CONCLUSION.....16

11. REFERENCES 18

1. INTRODUCTION

Finding the best route between two places is one of the most common applications of graph theory. A road network is naturally a graph: locations are ***vertices*** and roads are ***edges*** weighted by distance or time. Shortest-path and traversal algorithms on this graph power every modern navigation service. The ***Smart City Navigation System*** is a Flask web application that connects these ideas in two parts:

- ***Map Navigation (real world):*** search places, calculate real driving routes with alternatives, compare distance and estimated time, and save history and favourites (OpenStreetMap, Leaflet, Nominatim and OSRM).
- ***Algorithm Lab (ADS simulation):*** our own Python implementation of ***Dijkstra, BFS and DFS*** on a sample city graph, with step-by-step playback, road closures and re-routing.

The two parts deliberately use different data and engines: the Lab does not use the real map, and the real map does not use our Dijkstra code (OSRM has its own routing engine). The interface states this clearly, so the educational simulation is never mistaken for real navigation.

Technology stack: Python 3 and Flask (backend); SQLite (storage); HTML, CSS and JavaScript ES modules (frontend); Leaflet with OpenStreetMap tiles (map); Nominatim (place search); OSRM (road routing). ***Live project:*** https://smart-city-navigation-wwkj.onrender.com/?utm_source=chatgpt.com#map

PROBLEM STATEMENTS

Students usually meet Dijkstra, BFS and DFS only as static pseudo-code on small textbook graphs, and it is hard to see how they relate to a real navigation product. The problem addressed is to build a system that:

- finds the shortest route between two locations on a weighted graph and explains why it is
- shortest; shows how the answer changes when a road is closed, and detects when a destination
- becomes completely unreachable; provides real-world route planning (place search, alternatives, distance and time) in the same
- application, with an honest separation between simulation and real data; remembers user activity (history, favourites) and presents it as simple analytics.

2. OBJECTIVES

1 Model a city as a weighted, undirected graph using an adjacency list.

2. Implement Dijkstra's algorithm with a binary min-heap, and BFS and DFS using a queue and an explicit stack, in pure Python.
3. Visualise each algorithm step by step (settled order, relaxations, queue/stack state, path and cost).
4. Support dynamic road closures with automatic re-routing, before/after comparison and unreachable-node detection.
5. Integrate real-map navigation with place search, alternative routes, and distance/time comparison.
6. Persist history, favourites and algorithm runs in SQLite, and verify correctness with automated tests.
7. Deploy the application online for demonstration.

3. DATA STRUCTURE SELECTION AND JUSTIFICATION

The sample city has V = 9 locations and E = 14 roads (distances in km are sample values for teaching, not real measurements). Figure 1 shows the graph used by the Lab.

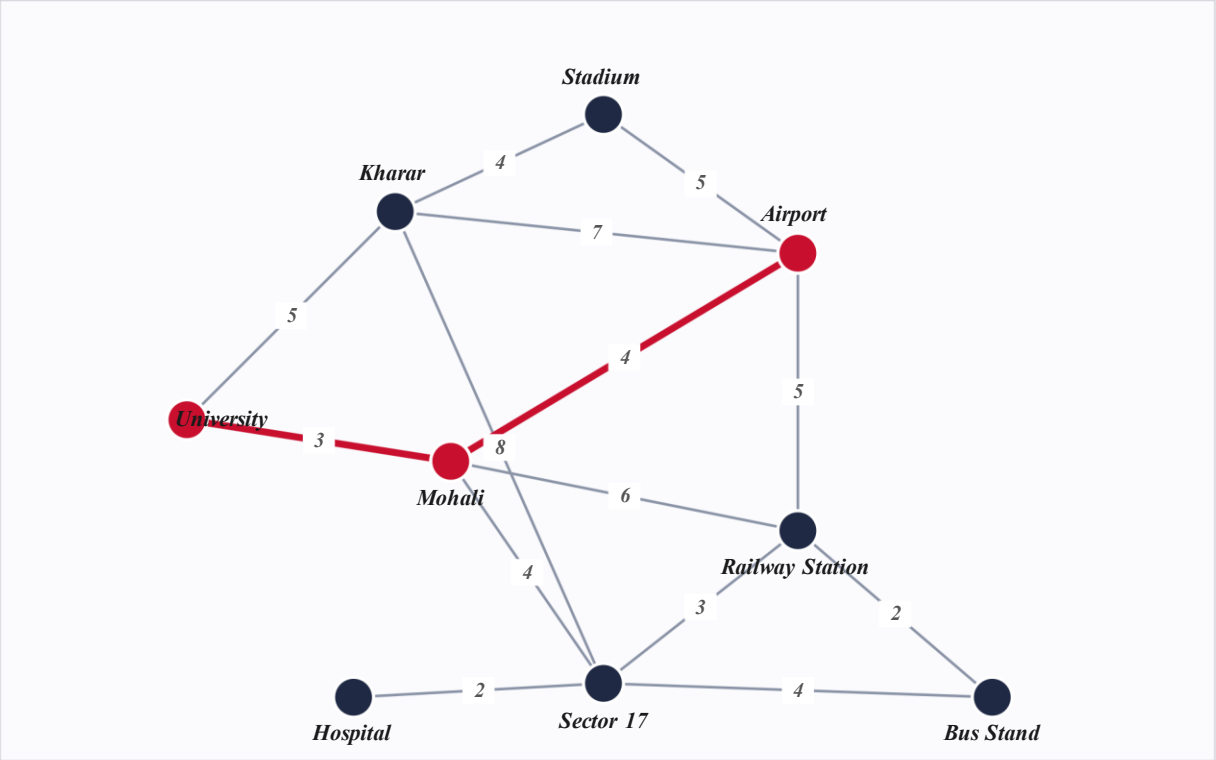


Figure 1: Sample city graph (edge labels in km). The highlighted path University - Mohali - Airport is the Dijkstra result (7 km).

Data structure	Where it is used	Why it was chosen
Weighted undirected graph (dict of dicts)	City map in graph_algorithms.py	O(V + E) space against O(V²) for a matrix; fast iteration over neighbours; O(1) edge lookup.
Binary min-heap (heapq)	Priority queue in Dijkstra	Extract-min in O(log V). No decrease-key in heapq, so a new entry is pushed and stale ones are skipped.
Queue (collections.deque)	Breadth-First Search	O(1) enqueue/dequeue gives correct level-bylevel order.
Stack (list of iterators)	Depth-First Search	Explicit stack avoids Python's recursion limit (tested on a 3000-node chain) with the same order as recursion.
Hash table (dict / set)	dist, prev, settled, visited	O(1) average look-up and update.

Set of sorted edge tuples		Closed roads	A road (a, b) and (b, a) map to one key; O(1) "is this road closed?" test.
Relational tables (SQLite)	History, favourites, runs	Persistent storage that survives restarts; algorithm parameterised queries prevent SQL injection.	

4. ALGORITHM AND FLOWCHART

4.1 Algorithm

A. Dijkstra's shortest path (non-negative weights; stops when the destination is settled)

1. Set dist[start] = 0 and dist[v] = INF for every other vertex; push (0, start) into a min-heap.
2. Pop the entry (d, u) with the smallest distance. If u is already settled or d > dist[u], it is stale: skip it.
3. Mark u as settled and record it in the visiting order. If u is the destination, stop.
4. For every ***open*** neighbour v of u with weight w: if d + w < dist[v], set dist[v] = d + w, prev[v] = u and push (dist[v], v).
5. Repeat from step 2. If the heap becomes empty and dist[end] is still INF, report "no route"; otherwise rebuild the path from prev[].

B. Breadth-First Search (queue): enqueue the start; repeatedly dequeue u, record it, and enqueue every unvisited open neighbour. It explores level by level and counts roads (hops), not kilometres.

C. Depth-First Search (stack): push the start; repeatedly look at the top vertex, move to its next unvisited open neighbour and push it; when none remains, pop (backtrack). Neighbours are taken in alphabetical order for reproducible results.

D. Road closure and re-routing: a closed road is added to a set of sorted edge keys. neighbors() skips closed roads, so every algorithm automatically sees the changed graph. If no route remains, connected components (BFS) identify the cut-off locations and each closed road is tested to suggest which one would reconnect them.

4.2 Flowchart

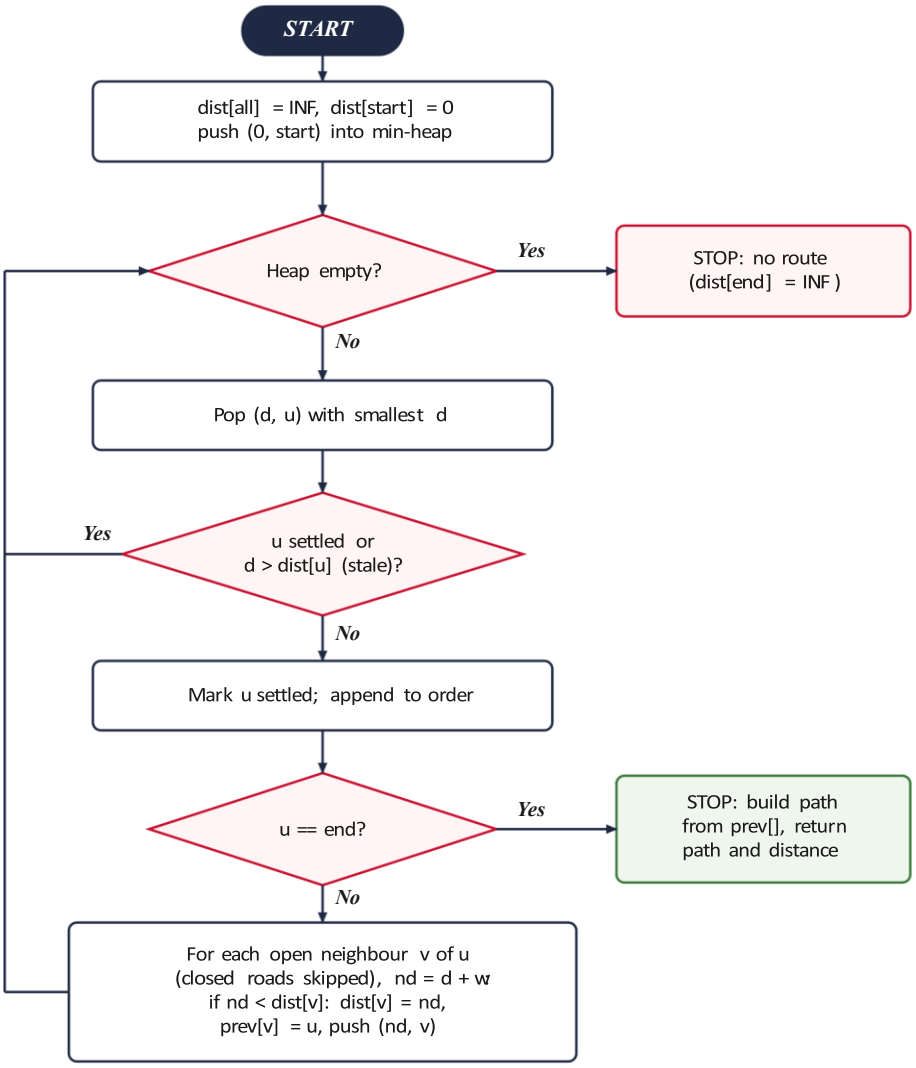


Figure 2: Flowchart of Dijkstra's algorithm as implemented in the project.

5. WORKING IMPLEMENTATION (PYTHON SOURCE CODE)

The backend follows a modular design (Figure 3). The algorithms are pure Python with no Flask or database imports, so they can be unit-tested and explained on their own. Core listings below are condensed from the project source (step-logging and validation details omitted).

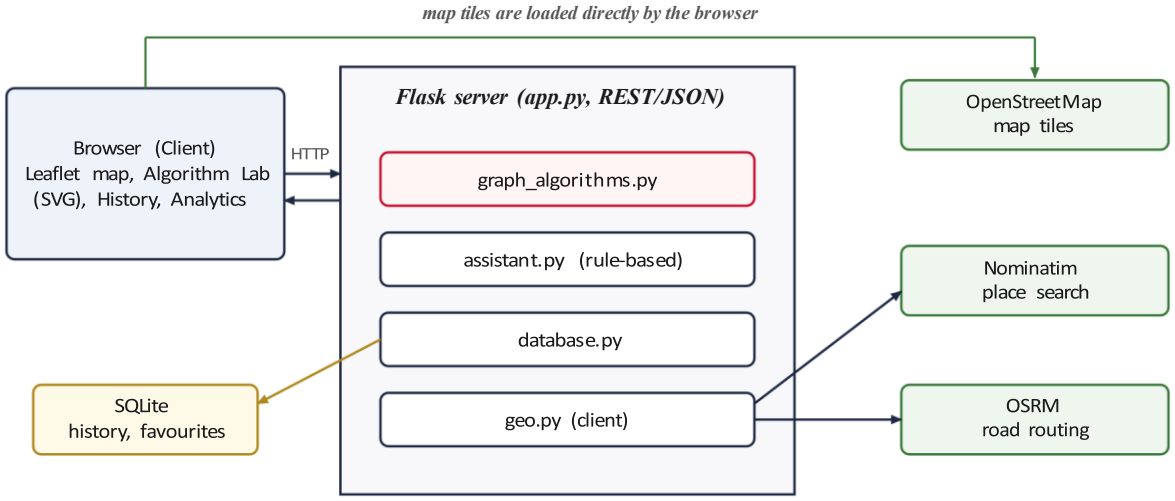


Figure 3: System architecture of the Smart City Navigation System.

File	Role
app.py	Flask application factory and all REST API routes; thread-safe closed-road state
graph_algorithms.py	CityGraph, dijkstra(), bfs(), dfs(), connected components
assistant.py	Rule-based explanations (no external AI used)
database.py	SQLite schema and parameterised queries
geo.py	Nominatim/OSRM client: User-Agent, 1 request/second throttle, cache, friendly errors
data/city_graph.json	Sample graph: 9 nodes, 14 edges, layout positions
templates/, static/	Single-page dashboard; JavaScript modules map, lab, history, analytics, ui, api, state
tests/	41 unit and API tests, plus an optional headless-browser test

5.1 *Graph data structure*

```
import heapq,math from
collections import deque

def edge_key(a, b):                              #canonical key of an undirected road
      return tuple(sorted((a, b)))

class CityGraph:
     def __init__(self, nodes, edges):
       self.adj = {n["id"]: {} for n in nodes}          # adjacency list
       for e in edges:          a, b, w = e["from"], e["to"], e["distance"]
       if w <= 0:
       raise ValueError("Edge weight must be positive")         self.adj[a][b]
= w
             self.adj[b][a] = w                              # undirected road

     defneighbors(self, node, closed=frozenset()):
         return[(v, w) for v, w in sorted(self.adj[node].items()) if
                 edge_key(node, v) not in closed]       # skip closed roads
```

5.2 Dijkstra's algorithm

```
def dijkstra(graph, start, end, closed=frozenset()):
```

```
       dist = {n: math.inf for n in graph.adj}
       prev, settled, order = {}, set(), []
       dist[start] = 0
       heap = [(0,
start)]        while
heap:

       d, u = heapq.heappop(heap)
       if u in settled or d > dist[u]:


            continue                                          # stale heap entry
 settled.add(u); order.append(u)
 if u == end:


            break                                             # destination settled
       for v, w in graph.neighbors(u, closed):
       if v not in settled and d + w < dist[v]:
       dist[v] = d + w          # relax edge (u, v)     prev[v] = u
       heapq.heappush(heap, (dist[v], v))      if dist[end] == math.inf:
       return {"reachable": False, "path": [], "distance": None, "order": order}
       path, cur = [end], end          while cur in prev:
    cur = prev[cur]; path.append(cur) return {"reachable": True, "path": path[::-1],
    "distance": dist[end], "order": order}
```

5.3 BFS, DFS and connected components

```
def bfs(graph, start, closed=frozenset()):
     seen, order = {start},[]
 queue = deque([start]) #FIFO queue  while queue:
       u = queue.popleft(); order.append(u)
       for v, _ in graph.neighbors(u, closed):
       if v not in seen:
                 seen.add(v); queue.append(v)


    return order

def dfs(graph, start, closed=frozenset()):
     seen, order = {start}, [start]
       stack = [(start, iter(graph.neighbors(start, closed)))] # explicit stack
       while stack:
       u, it = stack[-1]
       for v, _ in it:
       if v not in seen:
                 seen.add(v); order.append(v)
       stack.append((v, iter(graph.neighbors(v, closed))))     break
 else:


            stack.pop()                                       # dead end: backtrack
 return order

def components(graph, closed=frozenset()):     #used when no route exists
       seen, comps = set(), []         for s in graph.adj:     if s in seen:
       continue
       comp, q = [], deque([s]); seen.add(s)
       while q:
       u = q.popleft(); comp.append(u)         for
v, _ in graph.neighbors(u, closed):
 if v not in seen:
 seen.add(v); q.append(v)
 comps.append(sorted(comp))
 return comps
```

5.4 Database schema (SQLite)

```
CREATE TABLE IF NOTEXISTS route_history (   id INTEGER PRIMARY KEY AUTOINCREMENT,
origin_name TEXT NOT NULL, origin_lat REAL NOT NULL, origin_lon REAL NOT NULL,  REAL NOT
NULL,dest_name TEXT NOT NULL, dest_lat REAL NOT NULL, dest_lon
     distance_m REAL, duration_s REAL,
       alternatives INTEGER NOT NULL DEFAULT 1, created_at TEXT NOT NULL
);
```

```
CREATE TABLE IF NOT EXISTS favourites (
       id INTEGER PRIMARY KEY
AUTOINCREMENT,
 name TEXT NOT NULL, lat REAL NOT NULL, lon REAL NOT NULL, created_at TEXT NOT NULL );
CREATE TABLE IF NOT EXISTS algorithm_runs (
       id INTEGER PRIMARY KEY AUTOINCREMENT,
       algorithm TEXT NOT NULL, start_node TEXT NOT NULL, end_node TEXT,
       cost REAL, visited_count INTEGER NOT NULL, reachable INTEGER NOT
NULL,          runtime_ms REAL NOT NULL, created_at TEXT NOT NULL );
```

5.5 REST API

Endpoint	*Purpose*
GET /api/graph	Return the sample graph and closed roads
POST /api/algorithm	Run Dijkstra / BFS / DFS and return steps, path, cost, complexity
POST /api/road, /api/roads/reset	Close / reopen a road (re-runs the algorithm) / reopen all
GET /api/geocode?q=	Place search (Nominatim, server-side)
POST /api/route	Real driving routes with alternatives (OSRM)
GET|DELETE /api/history	List / clear route history
GET|POST /api/favourites	List / add favourites
GET /api/analytics	Statistics computed from stored data

6. SAMPLE INPUT AND OUTPUT

All outputs below were produced by running the project's own algorithm code on the sample graph.

6.1 Dijkstra: University to Airport

Step	*Settled node*	*Distance*	*Better distances found*
1	University	0 km	Kharar = 5, Mohali = 3
2	Mohali	3 km	Airport = 7, Railway Station = 9, Sector 17 = 7
3	Kharar	5 km	Stadium = 9
4	Airport	7 km	destination reached, stop

Output: path = University - Mohali - Airport; cost = ***7 km***; locations settled = 4; relaxations = 8.

6.2 Road closure and re-routing

Input	*Result*
University to Airport, all roads open	University - Mohali - Airport | 7 km
Close road ***Mohali - Airport***, re-run	University - Kharar - Airport | ***12 km*** (detour is 5 km longer)
Reopen the road (click it on the graph)	Original 7 km route is restored

6.3 Traversals from University

Algorithm	*Visiting order*
BFS (queue)	University - Kharar - Mohali - Airport - Sector 17 - Stadium - Railway Station - Bus Stand - Hospital University | Kharar, Mohali | Airport, Sector 17, Stadium, Railway Station | Bus Stand, Hospital
BFS levels	University - Kharar - Airport - Mohali - Railway Station - Bus Stand - Sector 17 - Hospital - Stadium
DFS (stack)	

6.4 Disconnected graph: University to Hospital

All roads open	University - Mohali - Sector 17 - Hospital | 9 km
	"No route available." Connected components: [Airport, Bus Stand, Kharar, Mohali, Railway Station, Sector 17, Stadium, University] and [Hospital].
Closethe only road ***Hospital - Sector 17***	Hospital is cut off; the assistant suggests reopening Hospital - Sector 17.

6.5 Real-map navigation (live site)

Example from the demonstration: searching "Chandigarh University" and "Chandigarh Airport" and pressing ***Find route*** draws the real route on the map, shows alternative routes (when OSRM returns them), labels the ***Fastest*** and ***Shortest*** options, and compares distance and estimated duration in a table. Times are free-flow estimates without live traffic. Figure 4 shows the Map Navigation screen of the live application (dark theme), with the route planner, route information panel, rule-based travel assistant and OpenStreetMap-based map.

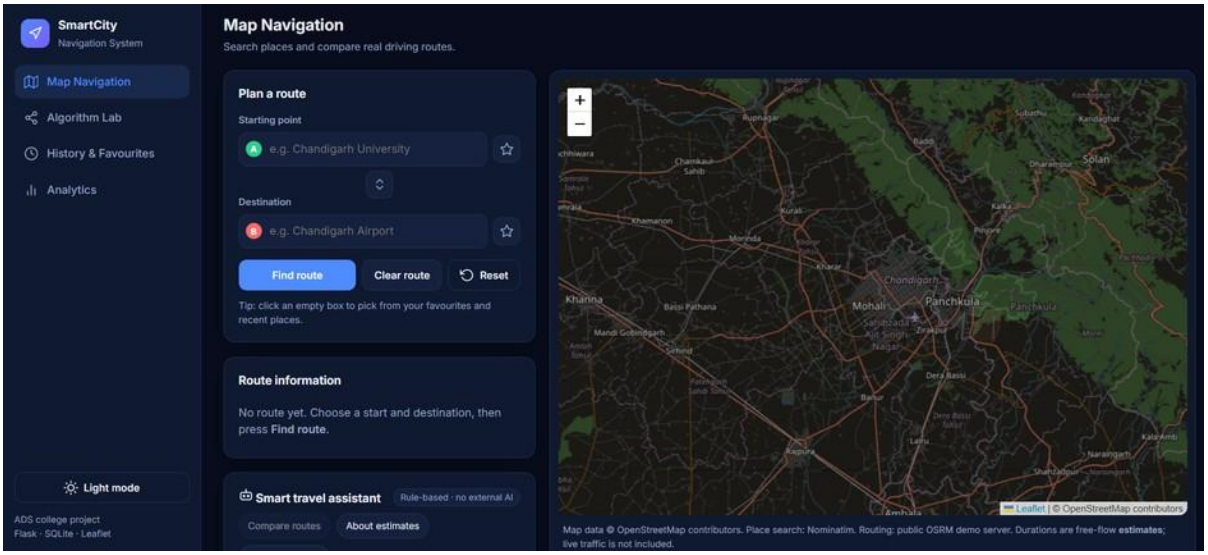


Figure 4: Map Navigation interface of the Smart City Navigation System (live application).

7. TIME COMPLEXITY ANALYSIS

Let V be the number of locations and E the number of roads.

Operation	*Time*	*Space*	*Remark*
Dijkstra (binary heap)	O((V + E) log V)	O(V)	$O(V^2)$ with a plain array instead of a heap
BFS	O(V + E)	O(V)	Each vertex enqueued once, each edge examined twice
DFS (explicit stack)	O(V + E)	O(V)	Depth of stack at most V
Connected components	O(V + E)	O(V) O(V	BFS from every unvisited vertex
Graph storage (adjacency list)	O(V + E)	+ E)	Matrix would need $O(V^2)$
Edge lookup / closed-road test	O(1) average	O(1)	Hash table / set
Close or reopen a road	O(1) + re-run	O(1)	Followed by the selected algorithm

Why a heap? Selecting the closest unsettled vertex costs O(V) in an array, giving $O(V^2)$ overall. A binary heap makes each selection O(log V). As heapq has no decrease-key, a vertex may be pushed more than once, but the cost stays O(E log V). ***Why non-negative weights?*** Once a vertex is settled its distance is final only if no later path can be shorter; a negative edge could break this, so the graph loader rejects non-positive weights.

Correctness check: Dijkstra is tested against an independent Bellman-Ford implementation on 200 random graphs with random closed roads.

8. PROJECT REPORT AND DEMONSTRATION

Live deployment: https://smart-city-navigation.onrender.com (hosted on Render; the first load may take a few seconds if the server was idle).

8.1 *Features*

- ***Map Navigation:*** live place suggestions, A/B markers, route line, alternative routes, Fastest/Shortest badges, comparison table, swap/clear/reset, clear error messages.
- ***Algorithm Lab:*** Dijkstra, BFS, DFS with play / step / back / finish, step log, SVG graph with weights, path highlight, complexity and beginner-friendly explanation.
- ***Road closures:*** click a road to close or reopen it; re-run, before/after comparison, reason shown, disconnected graphs handled.

- ***Assistant:*** rule-based (plain Python, no external AI) recommendations and explanations.
- ***History,favourites, analytics:*** SQLite with timestamps, reuse and delete; charts from stored data; dark/light theme.

8.2 *Testing*

Test suite	*Covers*	*Result*
python -m unittest discover -s tests -t .	Dijkstra (textbook and project graphs, vs Bellman-Ford), BFS levels, DFS order, disconnected graphs, closures, API validation, history and favourites, persistence after restart, analytics, network failure and no-route handling	*41 of 41 tests passed*
tests/e2e_browser.py (optional)	Headless-browser checks of the dashboard	Run separately with Playwright

8.3 Demonstration flow

1. Open the live site; on ***Map Navigation*** search two places, press ***Find route***, and compare the alternatives.
2. Switch to ***Algorithm Lab***; run Dijkstra from University to Airport (7 km) and use Step / Back to follow the settled order.
3. Close ***Mohali - Airport*** to see the 12 km re-route and the before/after box; reopen it by clicking the road.
4. Choose Hospital as destination and close ***Hospital - Sector 17*** to see “No route” and the cut-off nodes; run BFS and DFS.
5. Reuse and delete a history entry, then view the analytics dashboard.

8.4 Limitations

- OSRM durations are free-flow estimates with no live traffic; the public demo servers are shared and may be slow or rate-limited. Not for real navigation.
- Road closures in the Lab are kept in server memory and reset on restart; history and favourites persist. There are no user accounts, so data is shared on one server.
- The Lab graph is small and fixed (editable in data/city_graph.json); alternative routes depend on OSRM returning more than one route.

10. CONCLUSION

The Smart City Navigation System shows how core data structures turn into a working product: an adjacency-list graph, a binary min-heap, a queue and a stack drive Dijkstra, BFS and DFS; sets

and hash tables make road closures and lookups fast; SQLite adds persistence. The Algorithm Lab makes shortest-path reasoning visible, including re-routing and unreachable destinations, while the Map Navigation module places the same ideas in a real-world context. Automated tests (41 passing) support correctness, and the system is honest about what is simulated and what is estimated.

Future scope: user accounts, live-traffic weights, A* search on the real road graph, importing real city graphs, and weight-aware visual comparison of all three algorithms.

9. REFERENCES

1 T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, ***Introduction to Algorithms***, 4th ed., MIT Press, 2022.

2 E. W. Dijkstra, "A note on two problems in connexion with graphs," ***Numerische Mathematik***, vol. 1, pp. 269–271, 1959.

3 Python Software Foundation, "heapq: Heap queue algorithm" and "collections.deque," Python 3 documentation. https://docs.python.org/3/

4 Flask Documentation, Pallets Projects. https://flask.palletsprojects.com/

5 Leaflet: an open-source JavaScript library for interactive maps. https://leafletjs.com/

6 Project OSRM: Open Source Routing Machine. https://project-osrm.org/

7 Nominatim geocoding service and usage policy, OpenStreetMap. https://nominatim.org/

8 OpenStreetMap contributors. https://www.openstreetmap.org/

9 SQLite Documentation. https://www.sqlite.org/docs.html